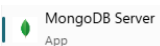




Chapter S3E: MongoDB Shell Exercise

This exercise introduces the basics of the MongoDB Shell, including database and collection creation, document manipulation, and data querying. While the MongoDB Shell commands differ slightly from PyMongo syntax used in the exam, this exercise focuses on building conceptual understanding rather than exam-specific commands.

A. Introduction to Mongo Shell

1. Start **MongoDB Server** (by clicking the icon  on school's laptop).
2. Start client application. Using the school's laptop, open PowerShell and change the path to **C:\Program Files\MongoDB\Server\3.4\bin** and type **.mongo**. The Mongo Shell should start and you should get a prompt like this:

```
Select C:\Program Files\MongoDB\Server\3.4\bin/mongo.exe
MongoDB shell version v3.4.9
connecting to: mongodb://127.0.0.1:27017
MongoDB server version: 3.4.9
Server has startup warnings:
2024-01-20T23:54:47.061+0800 I CONTROL [initandlisten]
2024-01-20T23:54:47.061+0800 I CONTROL [initandlisten] ** WARNING: Access control is not enabled for the database.
2024-01-20T23:54:47.061+0800 I CONTROL [initandlisten] ** Read and write access to data and configuration is unrestricted.
2024-01-20T23:54:47.061+0800 I CONTROL [initandlisten]
>
```

3. In the Mongo Shell console, type **2 + 2** and enter. You should get **4**.
4. Type **for (var i=1; i<=5; i++) { print(i); print("Hello");}** and press enter. You should get the **integers 1 to 5** and five **Hello** printed out.
5. Type **show dbs** and press enter. You should get a list of all the databases on the MongoDB server along with their sizes. There should be a **local**, which is used for storing information related to the operation of the MongoDB server itself.
6. Type **help** and you should get the following display.

```
C:\Program Files\MongoDB\Server\3.4\bin/mongo.exe
> help
db.help()                help on db methods
db.mycoll.help()          help on collection methods
sh.help()                 sharding helpers
rs.help()                 replica set helpers
help admin                administrative help
help connect              connecting to a db help
help keys                 key shortcuts
help misc                 misc things to know
help mr                   mapreduce

show dbs                  show database names
show collections           show collections in current database
show users                show users in current database
show profile              show most recent system.profile entries with time >= 1ms
show logs                 show the accessible logger names
show log [name]           prints out the last segment of log in memory, 'global' is default
use <db_name>             set current database
db.foo.find()             list objects in collection foo
db.foo.find( { a : 1 } )  list objects in foo where a == 1
it                         result of the last line evaluated; use to further iterate
DBQuery.shellBatchSize = x set default number of items to display on shell
exit                      quit the mongo shell
>
```

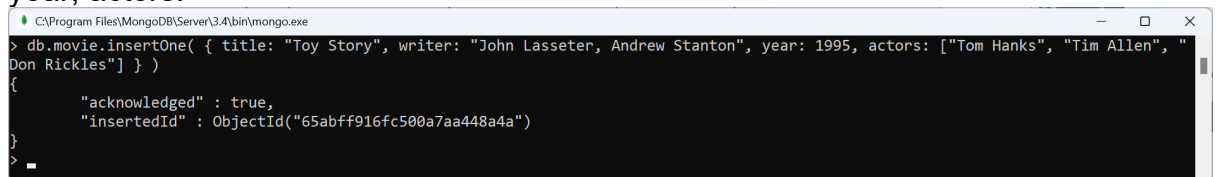
7. Type this in the shell: **"Hello, World!".replace("World", "MongoDB");**
8. Type this in the shell
>x=100 (then press enter)
>x/5 (then press enter).

B. Understanding MongoDB Databases and Collections

9. Create a MongoDB database named “**demo**” by typing **use demo**. You should see the prompt “switched to db demo”.
10. Type **show dbs** and press enter. Do you see the database “**demo**”?
11. Type **db.createCollection(“Restaurant”)** to create a new collection named “**Restaurant**”.
12. Now type **show dbs** and press enter. Do you see the database “**demo**”?
13. Type **show collections** to see the collections inside the database.
14. Type **db.Restaurant.drop()** to drop the collection. Show collections to see if “**Restaurant**” is deleted.
15. Type **db.dropDatabase()** to drop the database.

C. Understanding MongoDB Documents

16. Create a MongoDB database named “**entertainment**”.
17. Create a new collection named “**movie**”.
18. Check to see if the “**movie**” collection is inside the “**entertainment**” database.
19. Type **db.movie.insertOne({ title: "Toy Story", writer: "John Lasseter", year: 1995, actors: ["Tom Hanks", "Tim Allen", "Don Rickles"] })** to create a document in the “**movie**” collection. This document has 4 fields: title, writer, year, actors.



```
C:\Program Files\MongoDB\Server\3.4\bin>mongo.exe
> use entertainment
switched to db entertainment
> show collections
movie
> db.movie.insertOne( { title: "Toy Story", writer: "John Lasseter, Andrew Stanton", year: 1995, actors: ["Tom Hanks", "Tim Allen", "Don Rickles"] } )
{
  "acknowledged" : true,
  "insertedId" : ObjectId("65abff916fc500a7aa448a4a")
}
```

Notice an **ObjectId** is created with the document. How many bytes is the **ObjectId**?

20. Type these to define an array of **familyMovies**:

```
var familyMovies = [
  { title: "The Lion King",
    year: 1994,
    director: "Roger Allers, Rob Minkoff"},
  { title: "Frozen",
    writer: "Jennifer Lee",
    year: 2013,
    actors : ["Kristen Bell","Jonathan Groff"]},
  { title: "Finding Nemo",
    year: 2003,
    director: "Andrew Stanton"}
]
```
21. Type **db.movie.insertMany(familyMovies)** to insert multiple family movies into the “**movie**” collection.
22. Type **db.movie.find({})** to verify that that the documents are inserted.
23. Type **db.movie.find({}, {title: 1, year: 1})**. You should get documents with the fields **title** and **year** displayed.

24. Create a new collection named **"song"** and insert the following documents into the **"song"** collection.
- title: "Imagine", artist: "John Lennon", genre: "Pop", year: 1971
 - title: "What a Wonderful World", artist: "Louis Armstrong", genre: "Jazz", year: 1967
 - title: "Happy", artist: "Pharrell Williams", genre: "Pop", year: 2013
25. Verify that the documents are inserted correctly.

D. Manipulating MongoDB Documents in a Collection

26. Create a MongoDB database named **"company"**.
27. Write a MongoDB query to insert data into **"customer"** collection as shown:
- ```
db.customer.insertMany([
 {"name": "Midhuna", "age": 23, "place": "Amaravati" },
 {"name": "Akhil", "age": 24, "place": "New York" },
 {"name": "Honey", "age": 27, "profession": "Doctor" },
 {"name": "Manju", "age": 28, "place": "Vizag", "profession":
 "Driver" },
 {"name": "Bharat", "age": 24, "place": "New York", "profession":
 "Programmer" },
 {"name": "Arya", "age": 25, "profession": "Teacher" }])
```
28. Type **db.customer.find({}).pretty()** and see that the documents are indeed in the **"customer"** collection.
29. Type **criteria={name:"Midhuna"}** and press enter. It should display as **{"name": Midhuna }**.
30. Type **db.customer.find(criteria)** and it should display the document for Midhuna.
31. Type **update={name:"Midhuna",age:28,place:"Vizag"}** and press enter.
32. Type **db.customer.update(criteria,update)** and press enter.
33. Type **db.customer.find(criteria)** and you will find that Midhuna's place has been updated to Vizag.
34. Type **criteria={place:"New York"}** and press enter.
35. Type **db.customer.find(criteria).count()** and press enter. You should get a return of count 2.
36. Type **update={\$set:{bonus:250}}** and press enter.
37. Type **db.customer.update(criteria,update)** and press enter. You should get the following results: **{ "acknowledged" : true, "matchedCount" : 2, "modifiedCount" : 2 }**
38. Verify the update by typing **db.customer.find(criteria)**.
39. Type **db.customer.find({"age" : 24})** and it should return 2 documents.
40. Type **db.customer.deleteOne({"age" : 24})** and you should get the following **{ "acknowledged" : true, "deletedCount" : 1 }**
41. Count the number of document left by typing **db.customer.count()**.
42. Delete all documents from customers by typing **db.customer.deleteMany({})**.
43. Do a count on **"customer"** collection and you should get 0.

## E. Query Data in MongoDB

44. Go to PowerShell and type the following to import the database:

```
.\mongoimport --db demo --collection restaurant --file
```

```
D:\restaurants\restaurants.json.
```

- The database is named “**demo**” and the collection is named “**restaurant**”.
- The json file “**restaurants.json**” contains documents that will be imported into the MongoDB server.
- This json file is saved inside a folder called “**restaurants**” in D drive. If your json file is in H drive, then you should type **H:\restaurants.json** instead of **D:\restaurants\restaurants.json**

45. Type **show dbs** and make sure **demo** database is created.

46. Type **use demo**.

47. Type **show collections** and make sure **restaurant** collection is created.

48. Write a MongoDB query to display all the documents in the collection **restaurant**.

49. Write a MongoDB query to display the fields **restaurant\_id**, **name**, **borough** and **cuisine** for all the documents in the collection **restaurant**.

50. Write a MongoDB query to display the fields **restaurant\_id**, **name**, **borough** and **cuisine**, but exclude the field **\_id** for all the documents in the collection **restaurant**.

51. Write a MongoDB query to display the fields **restaurant\_id**, **name**, **borough** and **zip code**, but exclude the field **\_id** for all the documents in the collection **restaurant**.

52. Write a MongoDB query to display all the restaurant which is in the **borough Bronx**.

53. Write a MongoDB query to display the first 5 restaurant which is in the borough Bronx. Use **limit()** method with argument **5**.

54. Write a MongoDB query to display the next 5 restaurants after skipping first 5 which are in the borough Bronx. Use **limit()** and **skip()** methods with arguments **5**.

55. Write a MongoDB query to arrange the **name of the restaurants in ascending order** along with all the columns. Use **sort({name: 1})** method, where 1 means ascending order, and -1 means descending order.

56. Write a MongoDB query to arrange the **name of the restaurants in descending order** along with all the columns.

57. Write a MongoDB query to arrange the name of the cuisine in ascending order and for that same cuisine borough should be in descending order. Use **sort()** method, with arguments **{cuisine:1, borough:-1}**.